TOMAN SALES AI

Project Documentation

Mobile application, microservice backend, APIs, databases, security, provider boundaries, and offline demo workflow.

Submission overview

TOMAN is an owner-controlled AI sales workspace. Jarvis helps a business move faster while an owner keeps control of every consequential external action.

Included: React Native CLI mobile application, Spring Boot microservice backend, MariaDB/MySQL schemas, Gateway API inventory, provider boundaries, and local recording demo.

Project summary

TOMAN Sales AI is an owner-controlled AI sales workspace for small and growing businesses. Jarvis turns business context, customer data, and sales goals into reviewable plans. Jarvis can analyze, draft, recommend, and coordinate work, but an owner must review before any external action such as sending, calling, publishing, or spending budget.

The project includes a native Android mobile app and a Java/Spring Boot microservice platform. A local interactive demo is included for product recording; it shows sample metrics, leads, campaigns, agent activity, and integrations without contacting a backend or external provider.

Consolidated repository

```
TOMAN-Sales-AI/
backend/
platform/
discovery-service/ Eureka registry
config-service/ Central configuration
api-gateway/ Public API entry point and JWT filter
common-contracts/ Shared contracts
services/
auth-service/ Registration, verification, JWT, refresh session
workspace-service/ Workspace profile and settings
ingestion-service/ PDF/XLSX intake and metadata
jarvis-service/ Meta task orchestration and review outbox
notification-service/ Owner notification inbox
mobile-app/ React Native CLI application
docs/TOMAN_Project_Documentation.md
TOMAN_Sales_AI_Project_Documentation.pdf
```

Mobile UI technology

Area	Implementation
Runtime	React Native CLI 0.88.1 and React 19.3
Language	TypeScript
Platforms	Android now; iOS project included
UI	Native React Native components with a TOMAN design-token system
Secure session storage	`react-native-keychain`
File intake	`@react-native-documents/picker`
Voice	`react-native-tts` for offline recording/demo speech
Device layout	`react-native-safe-area-context`
Quality	ESLint, TypeScript, Jest

Bootstrap CSS is appropriate for a React desktop/web interface, but it does not run in native React Native. The shipped mobile app uses native UI components and consistent TOMAN design tokens.

Mobile journeys

1. Owner registration with name, business name, email, mobile number, password, and sales goal.
2. Email verification with numeric code, retry protection, and visible errors.
3. Business setup with industry, sales focus, and data sources.
4. Jarvis introduction explaining agent handoffs and owner approval controls.
5. Dashboard with revenue influence, qualified leads, replies, meetings, ROI, agent activity, reviews, and alerts.
6. Jarvis conversation with prompts, campaign proposals, delegated agents, document intake, and review flow.

7. Settings, notification preferences, secure sign-out, and device document selection.
8. Offline interactive demo: sample data only, no network request, with a spoken greeting: "Welcome to TOMAN AI. I am Jarvis, your AI sales manager. How can I help you today?"

Microservice architecture

All mobile requests enter through the API Gateway. Eureka supplies service discovery and Spring Cloud Config distributes service configuration. Every business capability owns its own MariaDB/MySQL schema and Flyway migrations.

```
React Native mobile app
|
v
API Gateway :8080 - JWT validation, routing, correlation IDs
|
+--- Eureka Registry :8761
+--- Config Service :8888
+--- Auth Service :8091 -------- toman_auth
+--- Workspace Service :8082 --- toman_workspace
+--- Ingestion Service :8086 --- toman_ingestion
+--- Jarvis Service :8085 ------ toman_jarvis
+--- Notification Service :8087 toman_notification
```

Service	Port	Responsibility
Discovery Service	8761	Eureka service discovery registry
Config Service	8888	Central local configuration distribution
API Gateway	8080	Single mobile entry point, JWT validation, routing, correlation IDs
Auth Service	8091	Registration, email verification, login, refresh tokens, logout, current owner
Workspace Service	8082	Business workspace profile, owner settings, trusted internal workspace lookup
Jarvis Service	8085	Meta Muse Spark task planning, agent lifecycle, review notification outbox
Ingestion Service	8086	PDF/XLSX upload, validation, storage metadata, secure downloads
Notification Service	8087	Owner review, permission, task-completion, and new-lead inbox

The Gateway reserves `/api/v1/reviews`, `/api/v1/approvals`, and `/api/v1/integrations` for future workflow and integration services. They are planned boundaries, not currently implemented modules.

Public API inventory

Local USB development uses Gateway base URL `http://127.0.0.1:8080` with `adb reverse tcp:8080 tcp:8080`. Protected endpoints require a bearer access token.

Auth Service - `/api/v1/auth`

Method	Endpoint	Purpose
POST	`/register`	Create pending owner and email-verification challenge
POST	`/verify-email`	Verify code and return authenticated session
POST	`/resend-verification`	Issue a fresh verification challenge
POST	`/login`	Authenticate owner

Method	Endpoint	Purpose
POST	`/refresh`	Rotate refresh session and access token
POST	`/logout`	Revoke active session family
GET	`/me`	Read authenticated owner profile

Workspace Service

Method	Endpoint	Purpose
POST	`/api/v1/workspaces`	Create workspace and default settings
GET	`/api/v1/workspaces/current`	Read current owner workspace
GET	`/api/v1/settings`	Read business and notification settings
PUT	`/api/v1/settings`	Update workspace settings
GET	`/internal/v1/workspaces/current`	Trusted service-to-service lookup only

Ingestion Service

Method	Endpoint	Purpose
POST	`/api/v1/documents`	Upload approved PDF or spreadsheet with multipart form data
GET	`/api/v1/documents`	List current workspace documents
GET	`/api/v1/documents/{documentId}`	Read document metadata
GET	`/api/v1/documents/{documentId}/download`	Download an authorized document

Jarvis Service

Method	Endpoint	Purpose
POST	`/api/v1/jarvis/tasks`	Create owner brief for Jarvis and Meta model
GET	`/api/v1/jarvis/tasks`	List workspace agent tasks
POST	`/api/v1/jarvis/tasks/{taskId}/owner-decision`	Record owner approval or rejection
GET	`/api/v1/jarvis/meta/status`	Safe Meta configuration status

Notification Service

Method	Endpoint	Purpose
GET	`/api/v1/notifications`	Paginated owner notification feed
POST	`/api/v1/notifications/{notificationId}/read`	Mark one notification read
POST	`/api/v1/notifications/read-all`	Mark the owner feed read
POST	`/internal/v1/notifications`	Trusted internal notification ingestion only

Database architecture

Each service uses a separate MariaDB/MySQL database. Flyway owns schema creation and migration versioning.

Database	Service	Tables
`toman_auth`	Auth	`user_accounts`, `email_verification_challenges`, `refresh_sessions`
`toman_workspace`	Workspace	`workspaces`, `workspace_settings`
`toman_ingestion`	Ingestion	`documents`
`toman_jarvis`	Jarvis	`agent_tasks`, `review_notification_outbox`
`toman_notification`	Notification	`owner_notifications`

Table	Purpose	Key data and controls
`user_accounts`	Owner identity	UUID, unique email/mobile, password hash, status, verification/login timestamps, optimistic-lock version
`email_verification_challenges`	Verification workflow	Hashed code, expiry, attempt count, consumed/invalidation state, owner foreign key
`refresh_sessions`	Secure mobile login	Hashed token, family ID, issue/expiry/revocation fields, replacement chain, device/IP context
`workspaces`	Business tenant profile	Owner ID, business name, industry, description, timezone, timestamps, version
`workspace_settings`	Owner preferences	Review, permission, completed-task, and new-lead notification switches
`documents`	Document metadata	Workspace/owner scope, filename, content type/category, byte size, SHA-256, storage path, status
`agent_tasks`	Jarvis lifecycle	Workspace/owner scope, prompt, state, model ID, plan JSON, failure detail, timestamps
`review_notification_outbox`	Reliable review handoff	Unique task, workspace/owner scope, attempts, errors, delivery state
`owner_notifications`	Owner inbox	Event type, title/message, optional reference, read state, created time

AI and provider APIs

Provider/API	TOMAN role	Current position
Meta AI Chat Completions / Muse Spark	Jarvis planning and specialist-agent reasoning	Implemented through Jarvis Service with backend-owned configuration
ElevenLabs Text to Speech	Premium Jarvis speech	Provider key and voice validated; server-side app adapter remains future work. Recording demo uses device TTS.
SMTP	Verification-code delivery	Auth Service capability configured through server-only environment variables
Meta Graph API / WhatsApp Cloud API	Consent-aware WhatsApp drafts and messages	Server-side configuration only; live adapter and webhook flow belong to future integration service
Facebook / Instagram Graph APIs	Draft posts, reels, and ad campaigns	Planned OAuth integration
Google Ads API	Campaign planning and performance insights	Planned OAuth integration
YouTube Data API	Video-ad and channel draft support	Planned OAuth integration

No mobile permission can replace provider OAuth authorization. Provider posting, messaging, calling, advertising, webhooks, and data collection must be executed server-side after consent and owner review.

Security and owner control

- Passwords and email verification codes are hashed.
- Refresh sessions are hashed, rotated, and revocable; mobile saves refresh state in the native secure keystore.
- Gateway validates JWTs before forwarding protected requests.
- Documents, tasks, notifications, and settings are owner- and workspace-scoped.
- Jarvis creates reviewable proposals and must not send, call, post, publish, or spend automatically.
- The notification outbox supports reliable review-message handoff.
- Keep API keys, JWT secrets, SMTP passwords, OAuth secrets, webhook secrets, and internal keys only in `backend/.env` or a production secret manager.
- Never place secrets in the mobile app, APK, JavaScript bundle, screenshots, or Git history.

Local development

```
cd backend
./scripts/start-local-stack.sh
```

```
cd mobile-app
npm install
npm start
npm run android -- --device <device-id>
```

For a USB Android phone:

```
adb reverse tcp:8080 tcp:8080
```

For the recording, tap **Open interactive demo** from Welcome or Sign in. The dashboard loads entirely from local sample data and Jarvis speaks through the device TTS engine.

GitHub handoff

Before a public push, confirm that `backend/.env`, runtime logs, build output, local database files, and provider credentials are ignored. `backend/.env.example` should contain variable names only. The mobile project retains its original nested `.git` directory to preserve history; deliberately migrate or remove that nested repository metadata before creating one parent Git repository.

Next production milestones: workflow-service, integration-service, OAuth callbacks, secure webhooks, object storage, monitoring, deployment secret manager, and backup strategy.